

Assignment 01 – Matrix Multiplication (GEMM)

Overview

This assignment implements **dense matrix multiplication** (General Matrix Multiply, GEMM) in C++ using two different algorithmic approaches:

1. **gemmSimple** – A straight-forward triple-nested $i/j/k$ loop implementation.
2. **gemmBlocked** – A cache-friendly, blocked (tiled) implementation that processes the matrices in fixed-size tiles/blocks to improve locality.

The task also includes a LeetCode-style test driver that automatically runs all input cases, validates the results against expected outputs, and times each approach.

Project Structure

```
assignment_01/
├── Makefile                # Build/run/clean targets
├── README.md               # This file
├── src/
│   ├── matmul.h            # Header declaring gemmSimple & gemmBlocked
│   └── matmul.cpp          # Implementations of both GEMM algorithms
├── driver/
│   └── driver.cpp          # LeetCode-style test runner & timing harness
└── tests/
    ├── inputs/             # test_01.txt ... test_08.txt (input matrices)
    └── outputs/            # test_01.txt ... test_08.txt (expected outputs)
```

Algorithms

gemmSimple

A textbook implementation of matrix multiplication.

- Input: matrix A of size $M \times K$, matrix B of size $K \times N$.
- Output: matrix C of size $M \times N$, where $C[i][j] = \sum_k A[i][k] * B[k][j]$.

- Uses three nested loops (i, j, k) in the natural order.
- **Time complexity:** $O(M * N * K)$.
- Simple and correct, but has poor cache behavior because it strides through B column-wise.

gemmBlocked

A blocked (tiled) version that improves cache locality.

- Processes the matrices in blocks of size $BLOCK = 32$.
- Loops iterate over blocks $(i0, j0, k0)$ first, then within each block over (i, j, k) .
- Block boundaries are clamped with $\min(...)$ to handle matrix sizes that are not exact multiples of the block size.
- **Time complexity:** Still $O(M * N * K)$, but with significantly better cache utilization for large matrices.
- Results are mathematically identical to `gemmSimple` (verified by the driver).

Build & Run

```
# Build the driver binary
make

# Build and run all test cases
make run

# Remove the built binary
make clean
```

The driver binary is produced at `driver/driver`.

Input / Output Format

Each test file under `tests/inputs/` uses the following format:

```
p q r
<A matrix, p rows of q integers>
<B matrix, q rows of r integers>
```

Where p q r are the matrix dimensions, A is $p \times q$, and B is $q \times r$.

For example, `test_01.txt`:

```
2 3 2
1 2 3
4 5 6
7 8
9 10
11 12
```

This describes **A** (2×3), **B** (3×2), and the expected output **C** (2×2):

```
58 64
139 154
```

Test Harness (**driver.cpp**)

The driver:

1. Scans `tests/inputs/` for all `.txt` files.
2. For each test, reads the input and the corresponding expected output from `tests/outputs/`.
3. Runs both `gemmSimple` and `gemmBlocked`, and times each with `std::chrono`.
4. Verifies that the two implementations produce identical results.
5. Compares the result against the expected output and reports `PASSED` / `FAILED`.
6. Prints a summary of total tests, passes, failures, and total time.

Running it produces output like:

```
Running 8 test(s)...
```

```
[1/8] test_01.txt -> PASSED | Time: 10.00 us)
...
```

```
===== SUMMARY =====
Total Tests : 8
Passed      : 8
Failed      : 0
Total Time  : 123.45 us
Result      : ALL TESTS PASSED
```

Configuration

- **Compilers/flags:** Defined in the `Makefile` (`g++` , `-O2` `-std=c++17` `-Wall` `-Wextra`).
- **Block size:** `BLOCK = 32` in `gemmBlocked` (adjustable inside `src/matmul.cpp`).

Dependencies

- A C++17 compiler (`g++` recommended).
- Standard Template Library (STL) – no external libraries required.